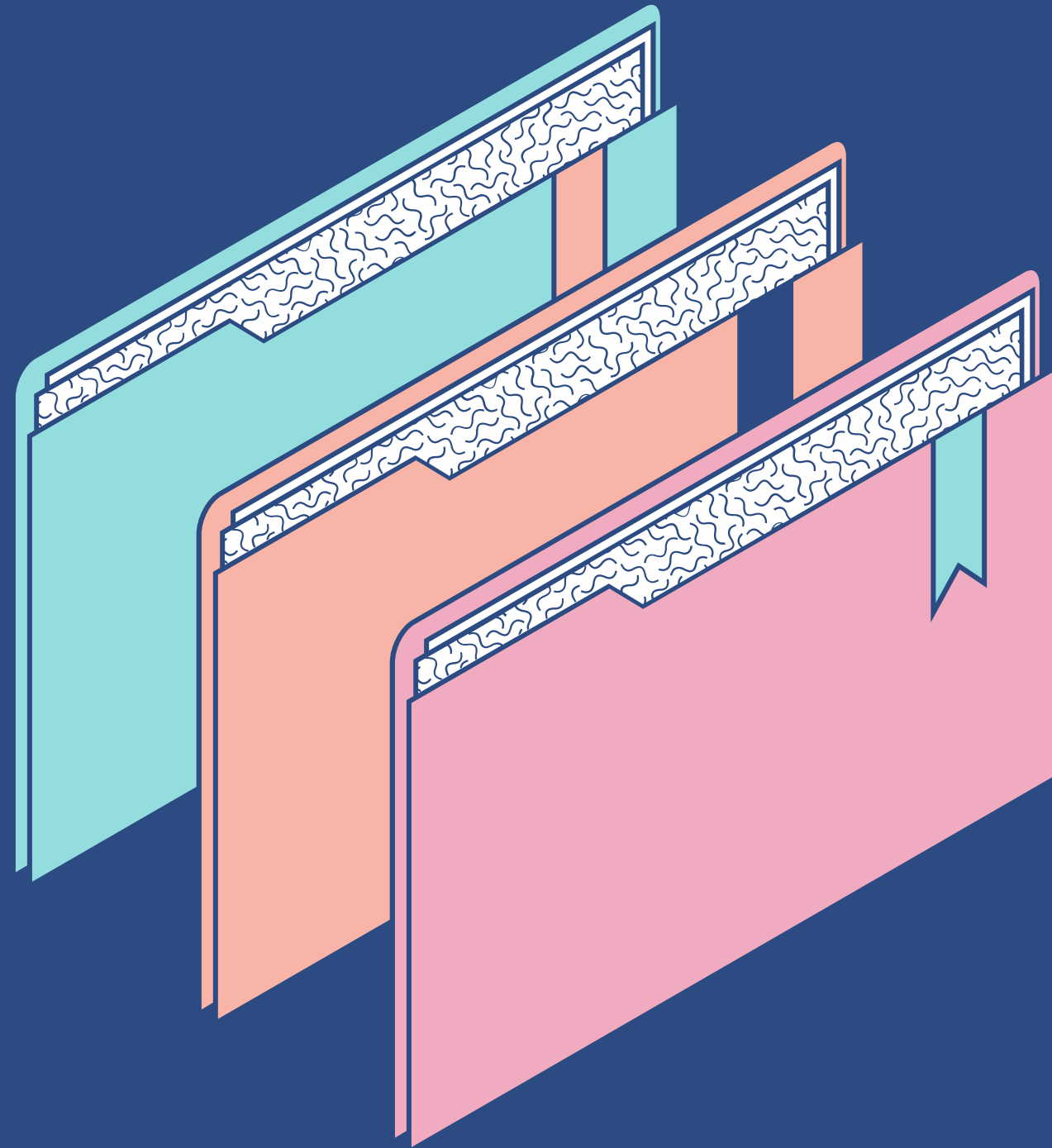




A QUICK INFORMATION GUIDE

Predictive Analytics: Text Classification

DS411 Predictive Analytics
1st Semester 2024-2025



Topics to tackle

KEY TOPICS DISCUSSED TO BE DISCUSSED

- Introduction to Text Classification
- Feature Extraction
- Classification Algorithms
- Evaluating Classification Models
- Remote learning and the new normal

Introduction to Text Classification

Text classification or categorization

involves trying to organize text documents into various categories based on inherent properties or attributes of each text documents.

involves the process of assigning text documents into one or more classes or categories.

Introduction to Text Classification



Types of text classification

- **Content-Based Classification:** This method focuses on what the text is actually about. It looks at key topics or words in the content to decide the category. For example, an article with words like "finance" or "stock market" would likely be classified under "Finance."
 - **Request-Based Classification:** This method tailors the categories based on what the user wants. It's designed to match specific interests or needs of the audience. For example, a news app might show tech news to users who usually search for technology topics.
-



AUTOMATE TEXT CLASSIFICATION

- Utilized ML techniques. Supervised ML or unsupervised ML
 - Extract meaningful pattern by grouping them together through similar data points. (clustering and summarize documents based on topic models)
 - Train/test dataset. Create a model, feed the training data to a model, then used the model to predict the class for future test data samples.
-

Types of Text Classification based on the number of Classes



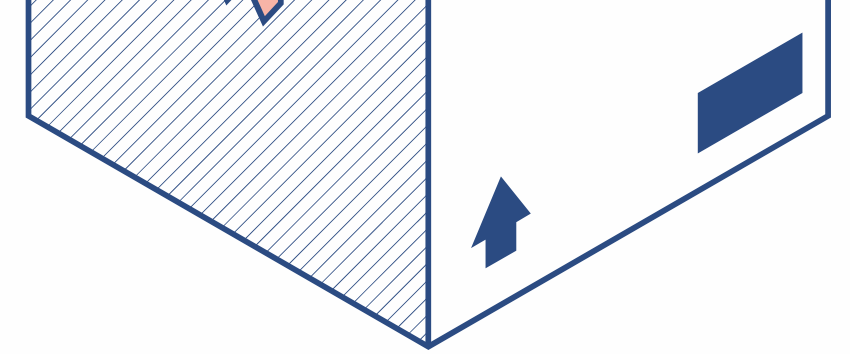
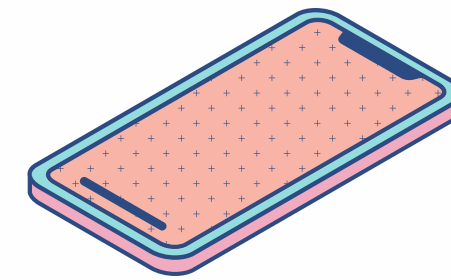
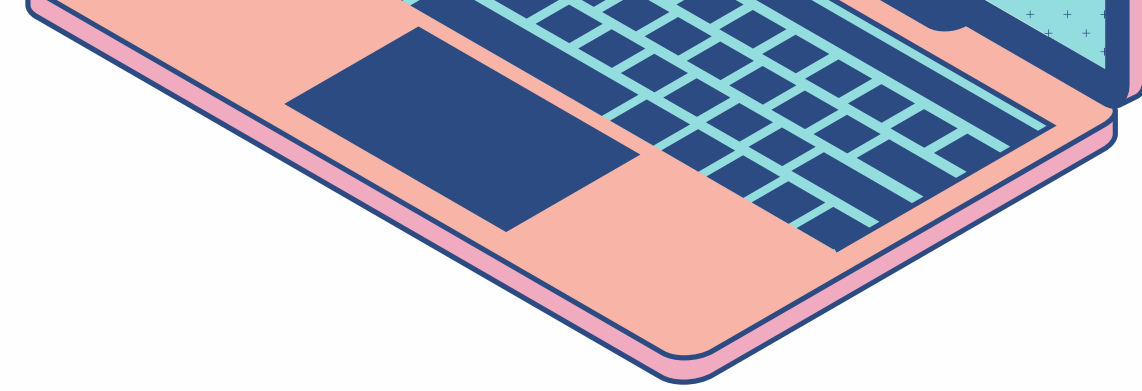
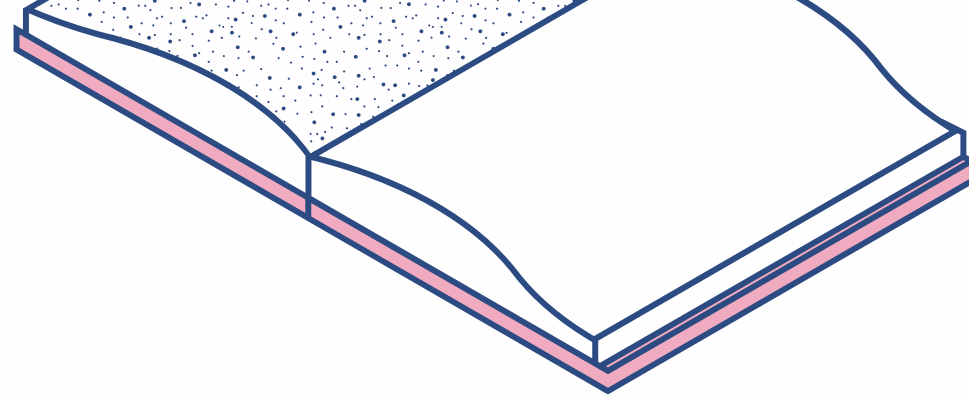
Types of text classification

- **Binary Classification** – total number of distinct classes or categories is two in number and any prediction can contain either one of those classes
 - **Multi-class classification** – also known as multinomial classification refers to a problem where the total number of classes is more than 2 and each prediction gives one class or category that can belong to any of those classes
 - **Multi-label classification** – each prediction can yield more than one outcome/predicted class for any data point
-



Feature Extraction

Bag of Words & TF-IDF

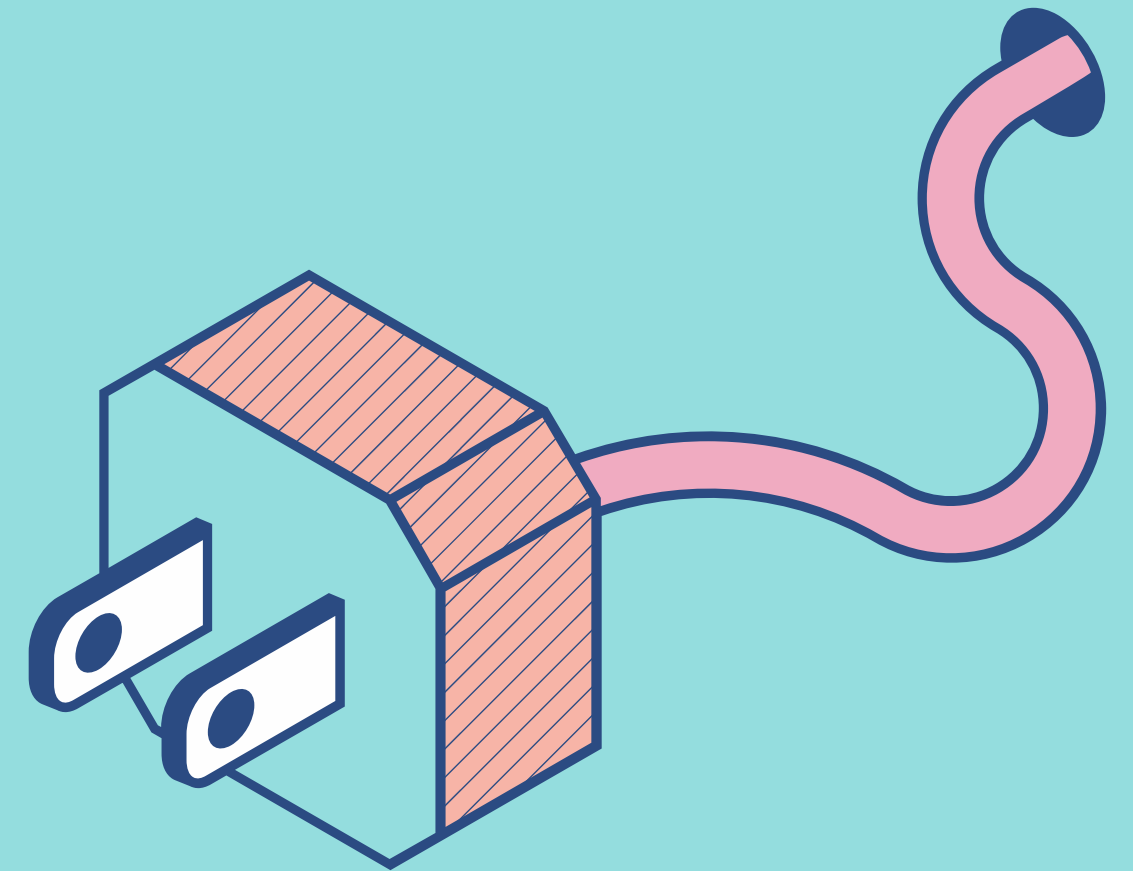


What are the Features in a Textual Data?

- Data points (rows and columns) – contains features or properties
- In ML – features are unique, measurable attributes or properties for each datapoint in a dataset. Features are numeric
- Extracted features are fed into ML algorithms for learning patterns that can be applied on future new data points

BAG OF WORDS(BOW) MODEL

- **BOW** convert text documents into vectors such that each document is converted into a vector that represents the frequency of all the distinct words that are present in the document vector space for that specific document.
- The weight for each word is equal to its frequency of occurrence in that document
- Occurrences of words
 - **N-grams** - continuous sequence of words or tokens in a document



N-GRAMS SHORT DETAILS

- N = 1, unigram
- N=2, bigram
- N=3, trigram
- N = n, n-gram
- Example: sentence = “I reside in CDO”
 - N=1, [“I”, “reside”, “in”, “CDO”]
- N = 2, [“I reside”, “reside in”, “in CDO”]
 - N=3, [“I reside in”, “reside in CDO”]

Bag of Words Implementation

Review 1: Game of Thrones is an amazing tv series!

Review 2: Game of Thrones is the best tv series!

Review 3: Game of Thrones is so great

In the table, I show all the calculations to obtain the Bag-Of-Words approach:

	amazing	an	best	game	great	is	of	series	so	the	thrones	tv
0	1	1	0	1	0	1	1	1	0	0	1	1
1	0	0	1	1	0	1	1	1	0	1	1	1
2	0	0	0	1	1	1	1	0	1	0	1	0

Bag of Words Implementation

```
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer
```

```
corpus = ['the sky is blue',
          'sky is blue and sky is beautiful',
          'the beautiful sky is blue',
          'i love blue cheese']
```

```
corpus
```

Bag of Words Implementation

```
v def bow_extractor(corpus, ngram_range=(1,1)):
    vectorizer = CountVectorizer(min_df=1, ngram_range=ngram_range)
    features = vectorizer.fit_transform(corpus)
    return vectorizer, features
```

```
bow_vectorizer, bow_features = bow_extractor(corpus)
```

```
features = bow_features.todense()
```

```
feature_names = bow_vectorizer.get_feature_names_out()
```

```
display(features, feature_names)
```

```
matrix([[0, 0, 1, 0, 1, 0, 1, 1],
        [1, 1, 1, 0, 2, 0, 2, 0],
        [0, 1, 1, 0, 1, 0, 1, 1],
        [0, 0, 1, 1, 0, 1, 0, 0]], dtype=int64)
array(['and', 'beautiful', 'blue', 'cheese', 'is', 'love', 'sky', 'the'],
      dtype=object)
```


- ***min_df***: Minimum Document Frequency. Sets a lower limit for a word to be included. If a word appears in too few documents, it gets ignored (to avoid rare, noisy words).
- ***max_df***: Maximum Document Frequency. Sets an upper limit for a word to be included. If a word appears in too many documents, it gets ignored (to avoid overly common words that don't help distinguish documents).

```
v def display_features(features, feature_names):  
    df = pd.DataFrame(data=features, columns=feature_names)  
    display(df)  
  
display_features(features, feature_names)
```

	and	beautiful	blue	cheese	is	love	sky	the
0	0	0	1	0	1	0	1	1
1	1	1	1	0	2	0	2	0
2	0	1	1	0	1	0	1	1
3	0	0	1	1	0	1	0	0

Bag of Words Implementation (using bi-grams)

```
def bow_extractor(corpus, ngram_range=(2,2)):
    vectorizer = CountVectorizer(min_df=1, ngram_range=ngram_range)
    features = vectorizer.fit_transform(corpus)
    return vectorizer, features
```

```
bow_vectorizer, bow_features = bow_extractor(corpus)
```

```
features = bow_features.todense()
```

```
feature_names = bow_vectorizer.get_feature_names_out()
```

```
matrix([[0, 0, 0, 0, 0, 1, 0, 1, 0, 1],
        [1, 0, 1, 0, 1, 1, 0, 2, 0, 0],
        [0, 1, 0, 0, 0, 1, 0, 1, 1, 0],
        [0, 0, 0, 1, 0, 0, 1, 0, 0, 0]], dtype=int64)
array(['and sky', 'beautiful sky', 'blue and', 'blue cheese',
      'is beautiful', 'is blue', 'love blue', 'sky is', 'the beautiful',
      'the sky'], dtype=object)
```

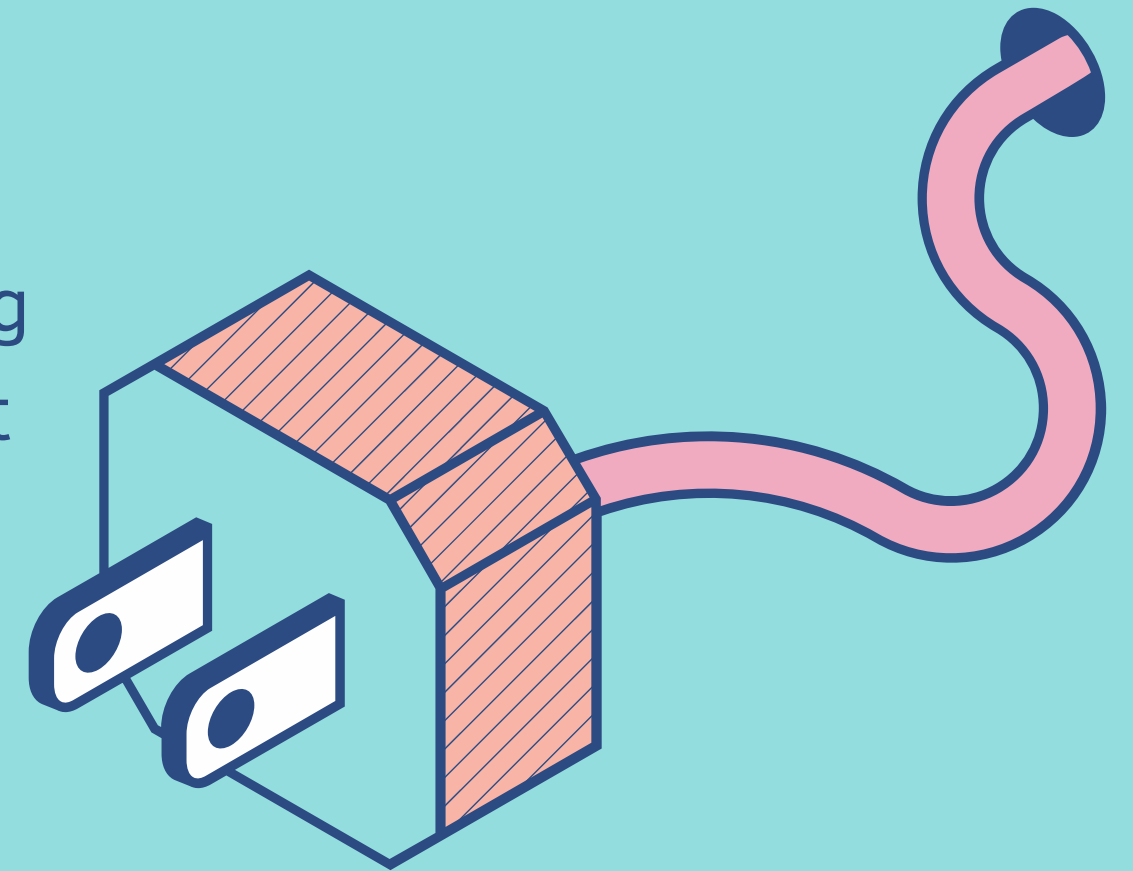
	and sky	beautiful sky	blue and	blue cheese	is beautiful	is blue	love blue	sky is	the beautiful	the sky
0	0	0	0	0	0	1	0	1	0	1
1	1	0	1	0	1	1	0	2	0	0
2	0	1	0	0	0	1	0	1	1	0
3	0	0	0	1	0	0	1	0	0	0

TF-IDF Model

Term-Frequency – Inverse Document Frequency

(It measures how important a word is in a specific document, considering how often it appears across all documents.)

- Originally developed as a metric for ranking functions for showing search engine results based on user queries it has come to be part of information retrieval and text feature extraction
- $Tfidf = tf \times idf$ where:
 - Tf – term frequency
 - Idf – inverse document frequency
- Tf = value acquired from BOW model
- Idf = divide the total number of documents in the corpus by the document frequency for each term and then apply logarithmic scaling on the result





1. **Term Frequency (TF):** Measures how often a word appears in a document.

Formula:

$$TF = \frac{\text{Number of times the word appears in the document}}{\text{Total number of words in the document}}$$

2. **Inverse Document Frequency (IDF):** Measures how unique a word is across all documents.

- If a word appears in many documents, its IDF score will be low (less unique).
- If a word is rare across documents, its IDF score will be high (more unique).

Formula:

$$IDF = \log \left(\frac{\text{Total number of documents}}{\text{Number of documents containing the word}} \right)$$

3. **TF-IDF:** Combines TF and IDF scores to highlight words that are **common in one document but rare across others**. Formula:

$$TF\text{-}IDF = TF \times IDF$$

Use TFIDFTRANSFORMER()

```
from sklearn.feature_extraction.text import TfidfTransformer  
import numpy as np
```

```
v def tfidf_transformer(bow_matrix):  
    t = TfidfTransformer(norm='l2', smooth_idf = True, use_idf=True)  
    tfidf_matrix = t.fit_transform(bow_matrix)  
    return t, tfidf_matrix
```

- Words with higher TF-IDF scores in each document are more distinctive to that document, meaning they carry specific meaning or relevance.
- Words with lower or zero scores are either not present in that document or are considered too common across documents to add unique value.
- By looking at which words have high TF-IDF scores, you can get a sense of each document's unique "themes" or important keywords.

```
def bow_extractor(corpus, ngram_range=(1,1)):
    vectorizer = CountVectorizer(min_df=1, ngram_range=ngram_range)
    features = vectorizer.fit_transform(corpus)
    return vectorizer, features
```

```
bow_vectorizer, bow_features = bow_extractor(corpus)
```

```
feature_names2 = bow_vectorizer.get_feature_names_out()
```

```
#Build tfidf transformer and show train corpus tfidf features
```

```
tfidf_trasnform, tfidf_features = tfidf_transformer(bow_features)
features_idf = np.round(tfidf_features.todense(), 2)
display_features(features_idf, feature_names2)
```

	and	beautiful	blue	cheese	is	love	sky	the
0	0.00	0.00	0.40	0.00	0.49	0.00	0.49	0.60
1	0.44	0.35	0.23	0.00	0.56	0.00	0.56	0.00
2	0.00	0.52	0.34	0.00	0.42	0.00	0.42	0.52
3	0.00	0.00	0.35	0.66	0.00	0.66	0.00	0.00

TO AVOID THE USE OF BOW, USE TFIDFVECTORIZER()

```
from sklearn.feature_extraction.text import TfidfVectorizer

def tfidf_extractor(corpus, ngram_range = (1,1)):
    vectorizer = TfidfVectorizer(min_df=1, ngram_range=ngram_range, norm='l2',smooth_idf=True, use_idf=True)
    features = vectorizer.fit_transform(corpus)
    return vectorizer, features

tfidf_vectorizer, tfidf_features = tfidf_extractor(corpus)

feature_names_tfidf = tfidf_vectorizer.get_feature_names_out()
features_tfidf = np.round(tfidf_features.todense(), 2)
display_features(features_tfidf, feature_names_tfidf)
```

	and	beautiful	blue	cheese	is	love	sky	the
0	0.00	0.00	0.40	0.00	0.49	0.00	0.49	0.60
1	0.44	0.35	0.23	0.00	0.56	0.00	0.56	0.00
2	0.00	0.52	0.34	0.00	0.42	0.00	0.42	0.52
3	0.00	0.00	0.35	0.66	0.00	0.66	0.00	0.00


```
file = './imdb_labelled.txt'
```

```
df = pd.read_csv(file, names=['sentence', 'label'], sep='\t')  
df
```

	sentence	label
0	A very, very, very slow-moving, aimless movie ...	0
1	Not sure who was more lost - the flat characte...	0
2	Attempting artiness with black & white and cle...	0
3	Very little music or anything to speak of.	0
4	The best scene in the movie was when Gerardo i...	1
...	...	...
743	I just got bored watching Jessica Lange take h...	0
744	Unfortunately, any virtue in this film's produ...	0
745	In a word, it is embarrassing.	0
746	Exceptionally bad!	0
747	All in all its an insult to one's intelligence...	0

748 rows × 2 columns

Binary Text Classification Example



Model Training using BOW

```
from sklearn.model_selection import train_test_split

x = df['sentence'].values
y = df['label'].values

X_train, X_test, y_train, y_test = train_test_split(X,y, test_size=0.25, random_state=101)
```

```
vectorizer = CountVectorizer()
X_train = vectorizer.fit_transform(X_train)
X_test = vectorizer.transform(X_test)
```

```
X_train
```

```
<Compressed Sparse Row sparse matrix of dtype 'int64'
      with 8647 stored elements and shape (561, 2563)>
```

Model Training using BOW

```
from sklearn.linear_model import LogisticRegression  
  
log_reg = LogisticRegression()  
log_reg.fit(X_train, y_train)  
score = log_reg.score(X_test, y_test)  
  
print(f'Accuracy score: {score}')
```

```
Accuracy score: 0.7754010695187166
```


Model Training using TFIDF

```
vectorizer_tfidf= TfidfVectorizer(min_df=1,  
                                   ngram_range=(1,1),  
                                   norm = 'l2',  
                                   smooth_idf= True,  
                                   use_idf=True)  
X_train = vectorizer_tfidf.fit_transform(X_train)  
X_test = vectorizer_tfidf.transform(X_test)
```

```
log_reg = LogisticRegression()  
log_reg.fit(X_train, y_train)  
score = log_reg.score(X_test, y_test)
```

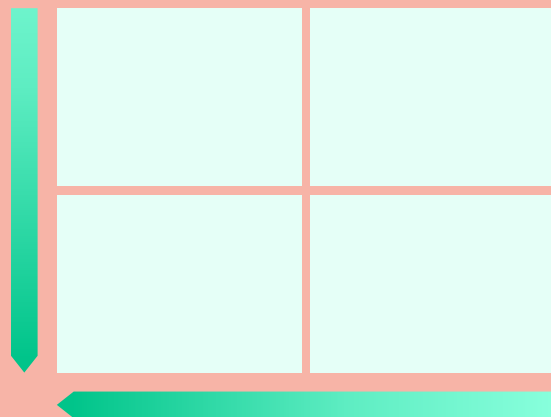
```
print(f'Accuracy score: {score}')
```

```
Accuracy score: 0.786096256684492
```

PERFORMANCE EVALUATION

1

CONFUSION MATRIX



2

PRECISION

the ratio
between the
True Positives
and all the
Positives

3

RECALL

the measure of
our model
correctly
identifying True
Positives

4

F1-SCORE

F1-score is the
Harmonic mean of
the Precision and
Recall

5

ACCURACY

the ratio of the total
number of correct
predictions and the
total number of
predictions

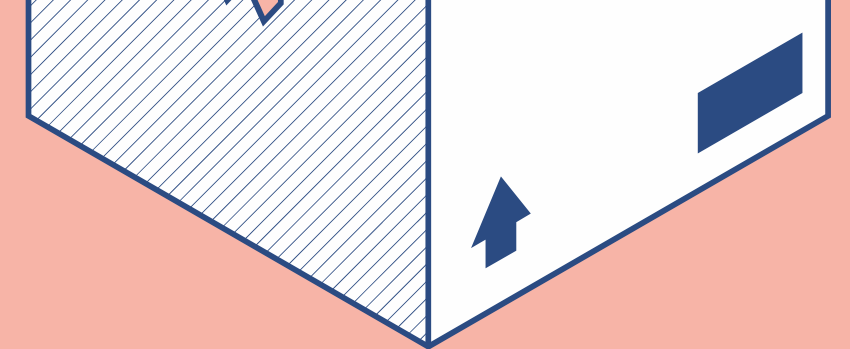
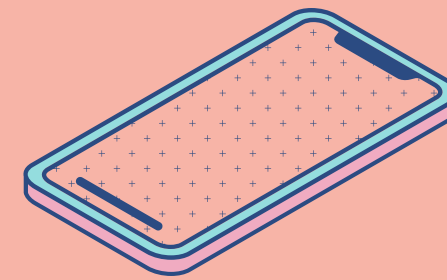
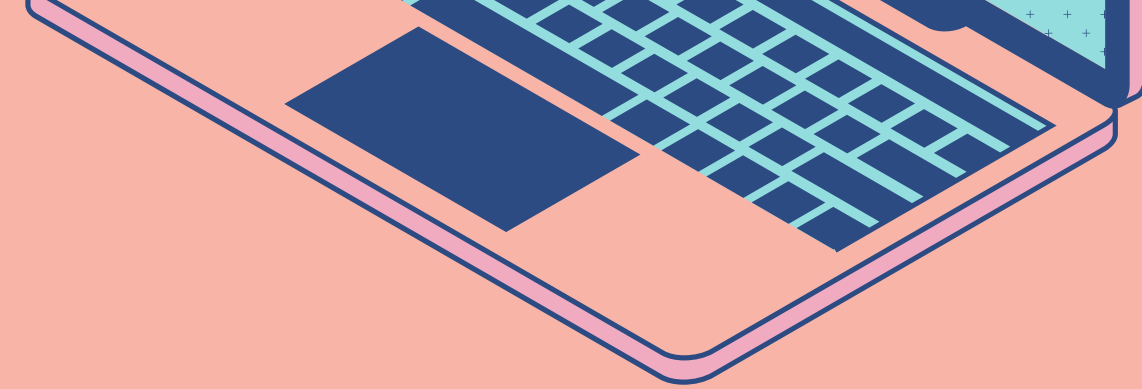
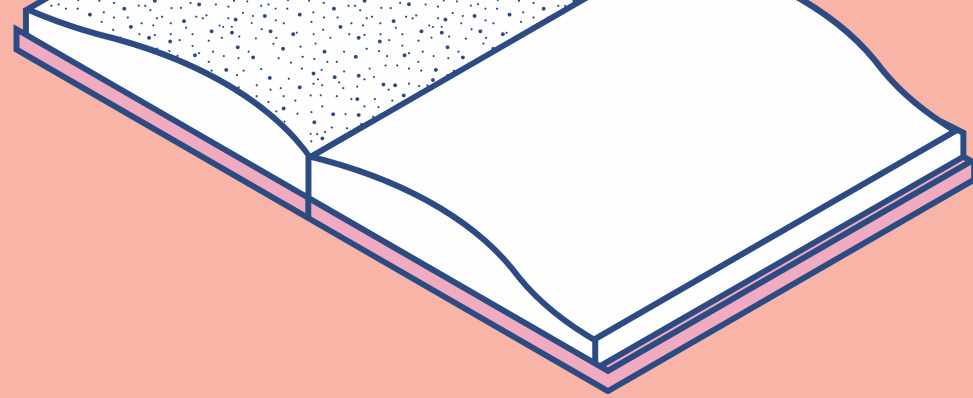


```
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report

y_pred = log_reg.predict(X_test)
```

```
confusion_matrix(y_test, y_pred)
pd.crosstab(y_test, y_pred, rownames=['Actual'], colnames=['Predicted'], margins=True)
```

Predicted	0	1	All
Actual			
0	64	26	90
1	14	83	97
All	78	109	187



```
print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	0.82	0.71	0.76	90
1	0.76	0.86	0.81	97
accuracy			0.79	187
macro avg	0.79	0.78	0.78	187
weighted avg	0.79	0.79	0.78	187

home work >

- **Implement the code above**
- **Dataset:** *amazon_cells_labelled.txt* dataset
- Get the *accuracy, confusion matrix, precision, recall, and F1 score* using the features extracted from both BOW and tfidf features

home work>

- Display the first 5 rows of the dataset
 - Display the number of rows and columns
 - Display the number of rows and columns for the **Bag-of-Words** extracted feature from the dataset
 - Display the feature names from the location 100 to 500
 - Perform text classification for Bag-of-Words
- ## Classification
- Define x and y matrix
 - Train and test the dataset using a test size of .0.25, random state of 101
 - Use Linear Regression Model
 - Get accuracy, precision, recall, f1 score
 - Display the confusion matrix

